الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري
Arab Academy for Science, Technology & Maritime Transport
College of Computing and Information Technology

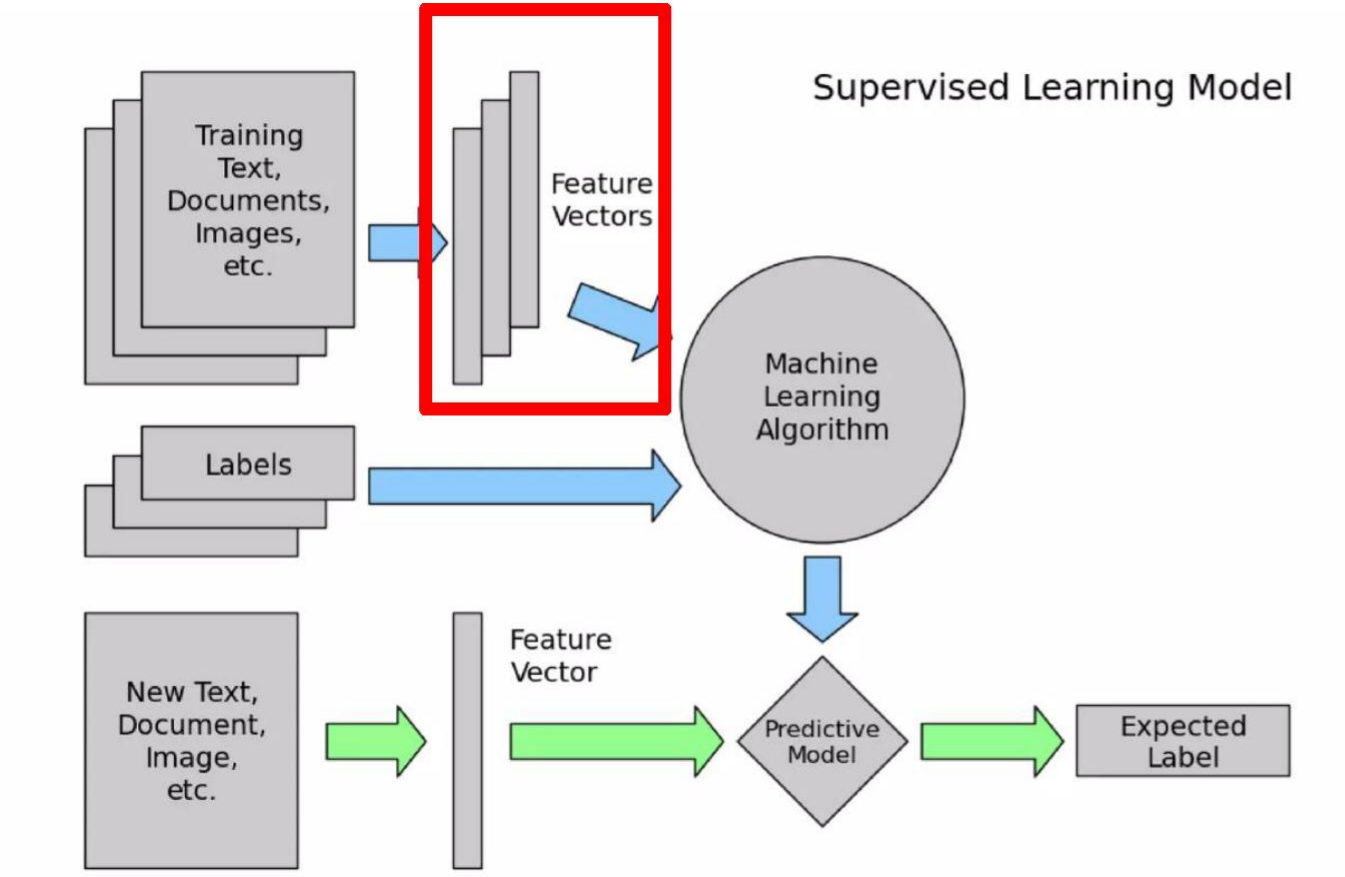
CAI3103/CS460

DEEP LEARNING

LECTURE-3&4

Convolutional Neural Networks (CNNs)

Traditional Machine Learning (ML) approach



Feature extraction:

- Requires expert knowledge
- Time-consuming, hand-tuning
- Mostly problem specific

Feature <u>learning</u> is required.

Traditional vs Deep Learning (DL)

Feature extractors, required:

► Prior Expert knowledge

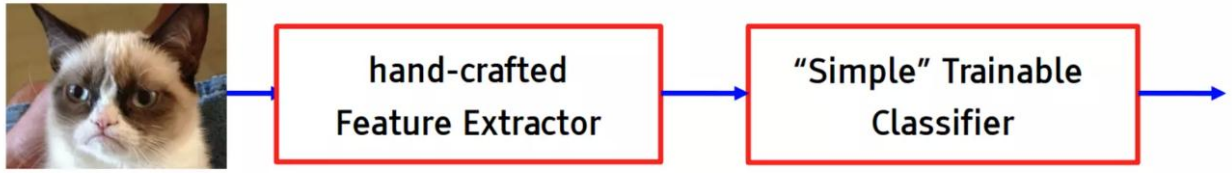
► Time-consuming manual parameter-tuning

► Problem-specific in most cases.

→ Using DL can resolve these challenges through (learning feature extractors)

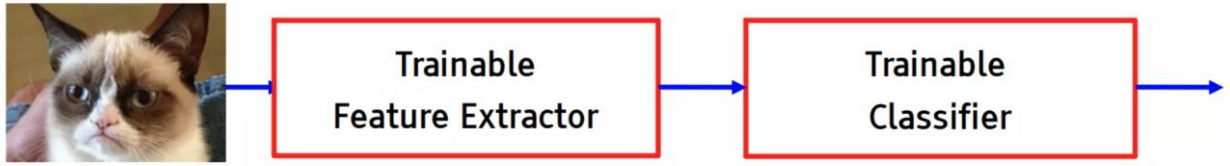
Traditional Approach

The traditional approach uses fixed feature extractors.



Deep Learning Approach

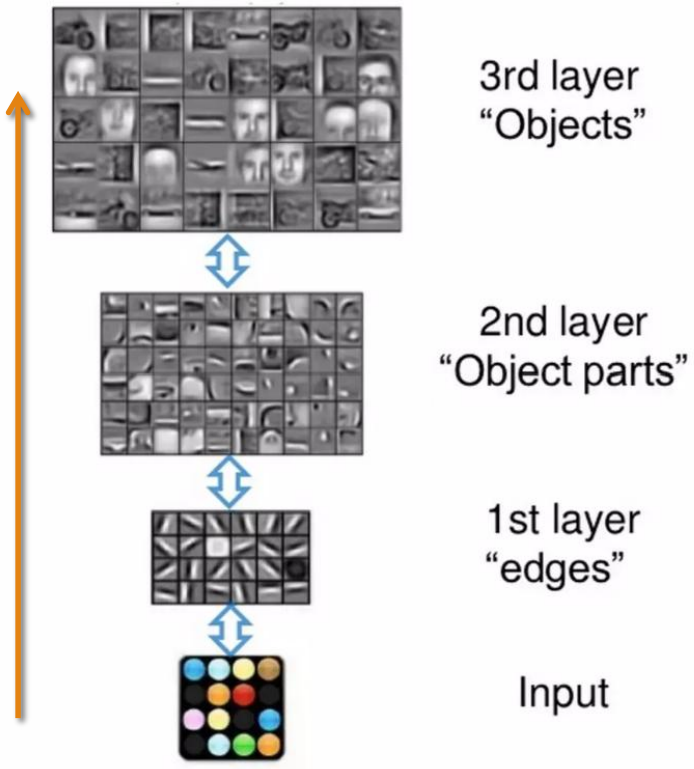
Deep Learning approach uses trainable feature extractors.



Traditional vs Deep Learning (DL)

- The hierarchical structure is common in DL Convolutional Neural Networks (CNNs), which are widely used for image recognition and other tasks.

- A bottom-up approach to feature extraction shows that how the network learns to identify increasingly complex features from the input data, as simpler features are combined to form more complex ones.



Source: Lee et.al., ICML2009

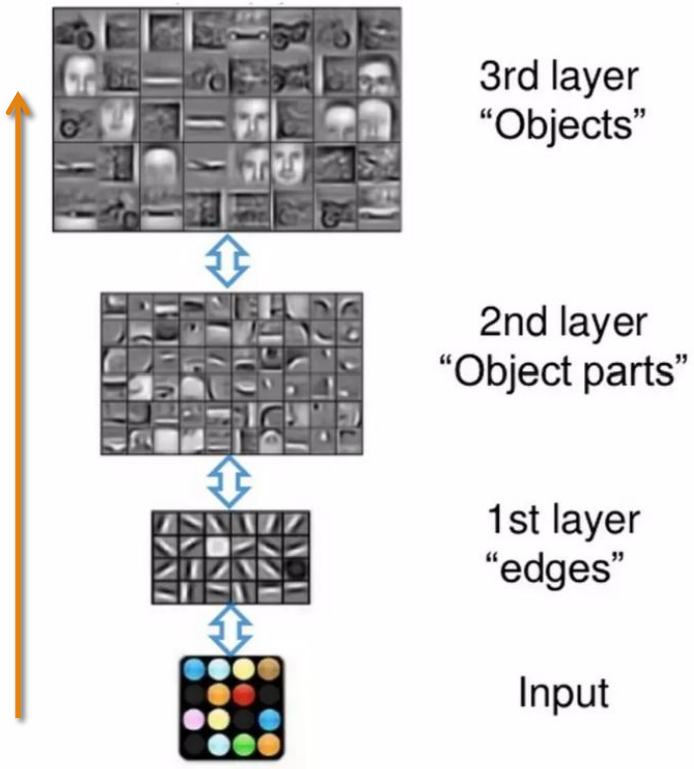
Traditional vs Deep Learning (DL)

Input: The bottom layer represents the input data.

1st Layer (Edges): This layer extracts basic features like edges and lines from the input. The output is shown as a grid of grayscale images, with darker areas indicating the presence of edges.

2nd Layer (Object Parts): This layer combines the edge features to detect more complex features, such as object parts or components. The output is a grid of grayscale images, but the patterns are more complex, suggesting the presence of specific object parts.

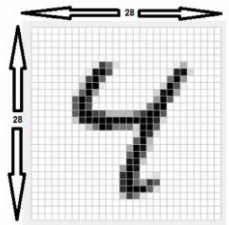
3rd Layer (Objects): This layer combines the object parts to identify complete objects. The output shows a grid of images, each representing a different object category.

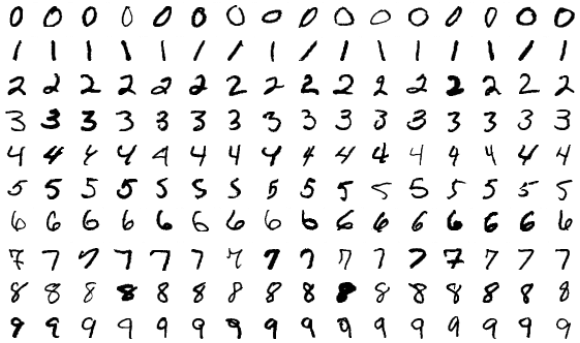


Source: Lee et.al., ICML2009

Benchmark Datasets for Computer Vision

MNIST (Modified National Institute of Standards and Technology) is a widely used handwritten digit classification dataset. It contains 60,000 training images and 10,000 testing images, each 28x28 pixels grayscale.





ImageNet is a much larger and more complex dataset compared to MNIST. It consists of over 14 million images organized into 22,000 categories. These images depict a variety of objects and scenes from the real world. ImageNet is frequently used for object detection, image classification, and other computer vision tasks.

Key changes from Machine Learning (ML) to Deep Learning (DL):

Architecture: DL introduced neural networks with multiple hidden layers, while ML often used simpler models like linear regression and decision trees.

Feature Engineering: DL automatically learns features from raw data, reducing the need for manual feature engineering in ML.

Scalability: DL can handle large datasets and complex tasks more effectively due to its ability to learn hierarchical representations.

Performance: DL has achieved significant improvements in accuracy and performance on various tasks compared to traditional ML methods.

Applications: DL has enabled breakthroughs in areas like computer vision, natural language processing, and speech recognition.

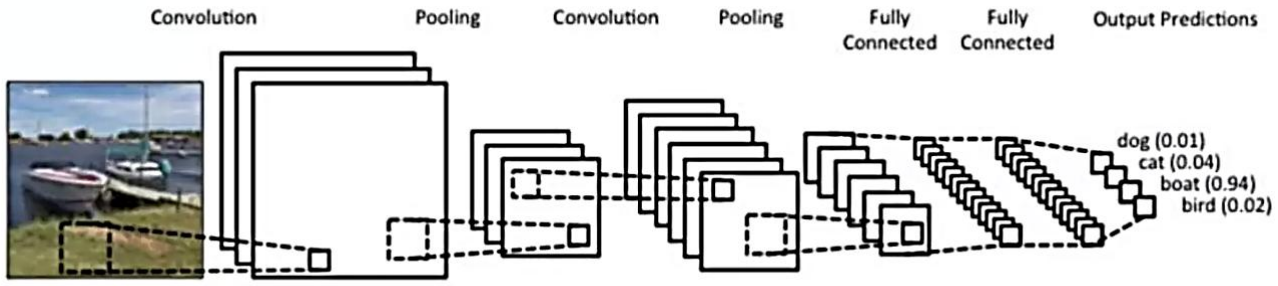
Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs)

- In the field of machine learning, a **Convolutional Neural Network (CNN)** is a class of deep, feed-forward artificial neural networks specifically designed for processing structured arrays of data, such as images.
- CNNs have become the standard for analyzing visual imagery due to their ability to automatically and adaptively learn spatial hierarchies of features.

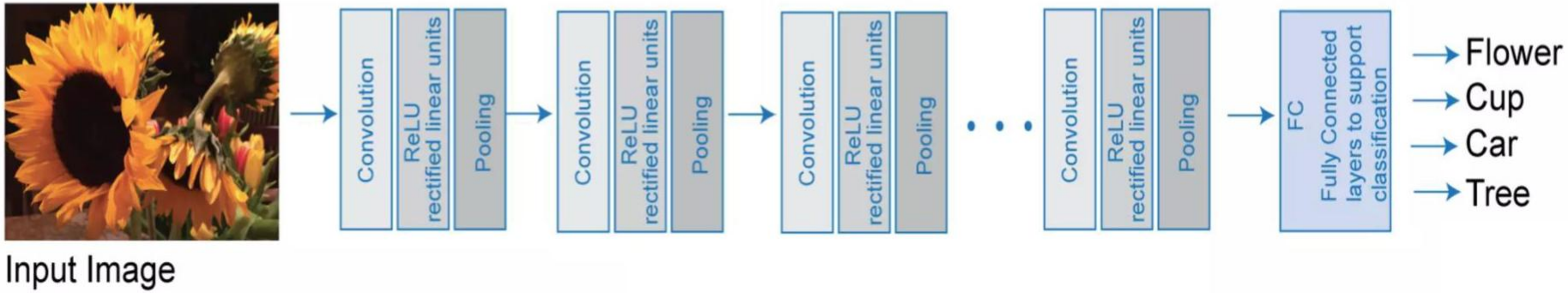
Deep Learning Applications in Computer Vision

- CNNs are the backbone of modern object detection, image segmentation, and facial recognition systems.



CNN Architecture

- **Convolutional Neural Network (CNN)**is a multi-layer neural network
- **Convolutional Neural Network** is comprised of one or more **convolutional layers** (often with a **pooling layers**) and then followed by one or more **fully connected layers**.

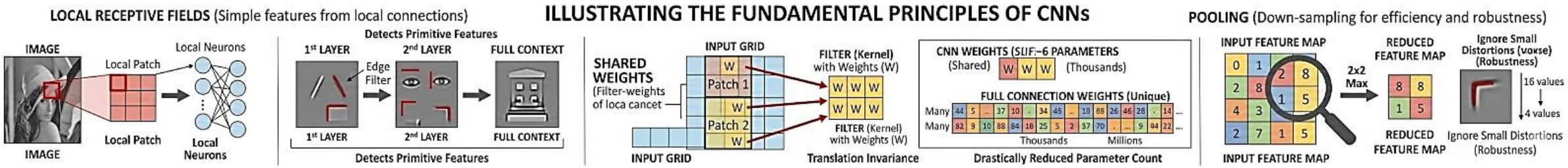


CNN Architecture

- Convolutional Neural Networks (CNNs) are based on three foundational concepts that differentiate them from traditional fully connected networks.
- These concepts allow CNNs to efficiently process data with a grid-like topology, such as images.

Fundamental Principles of CNNs:

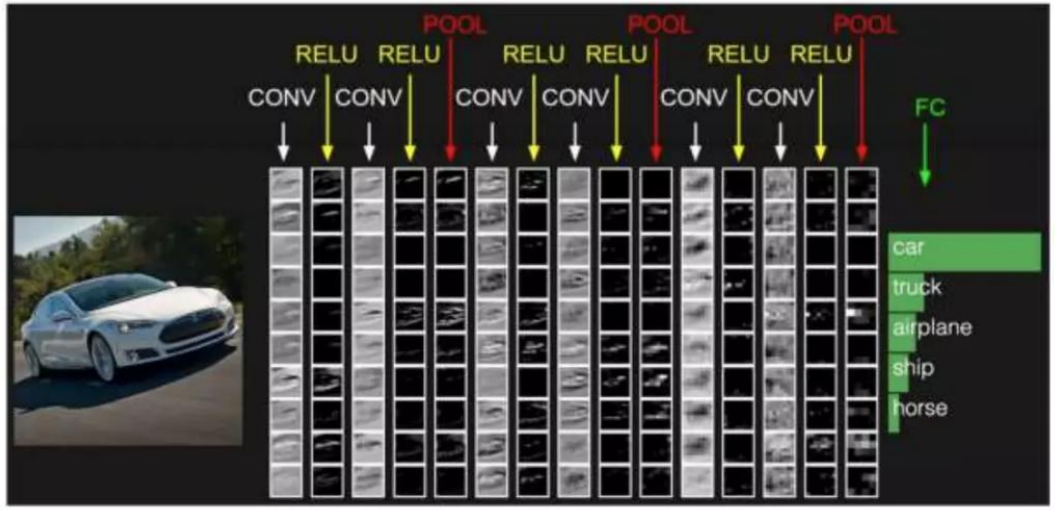
- **Local Connections (Receptive Fields):** Neurons connect only to localized input regions/patches (instead of looking at the whole image at once) to detect primitive/ simple features like edges and textures.
- **Shared Patterns (Weights):** The same filter (kernel/detector) is applied across the entire input image, which helps to recognize patterns no matter where it appears and keeps the model small and efficient.
- **Pooling:** Reduces spatial dimensions (shrinks the data) by keeping only the most important information to decrease computational overhead and helps it ignore minor shifts or "noise" in the image.



CNN Components

There are four main components in the CNN:

1. **Convolutional Layers** (Feature extraction)
2. **Activation Functions** (Non-linearity, e.g., ReLU)
3. **Pooling Layers** (Dimension reduction)
4. **Fully Connected Layers** (Final classification)



CNN Components

1. Convolution

- **Action: Filters (kernels) slide across the input image.**
- **Purpose: Extracts spatial features such as edges, textures, and patterns.**

2. Non-Linearity (ReLU)

- **Action: Applies an activation function (most commonly the Rectified Linear Unit (ReLU) $f(x) = max(0, x)$).**
- **Purpose: Introduces non-linear properties to the system, allowing it to learn complex mappings that a simple linear model cannot learn.**

3. Pooling (Sub-sampling)

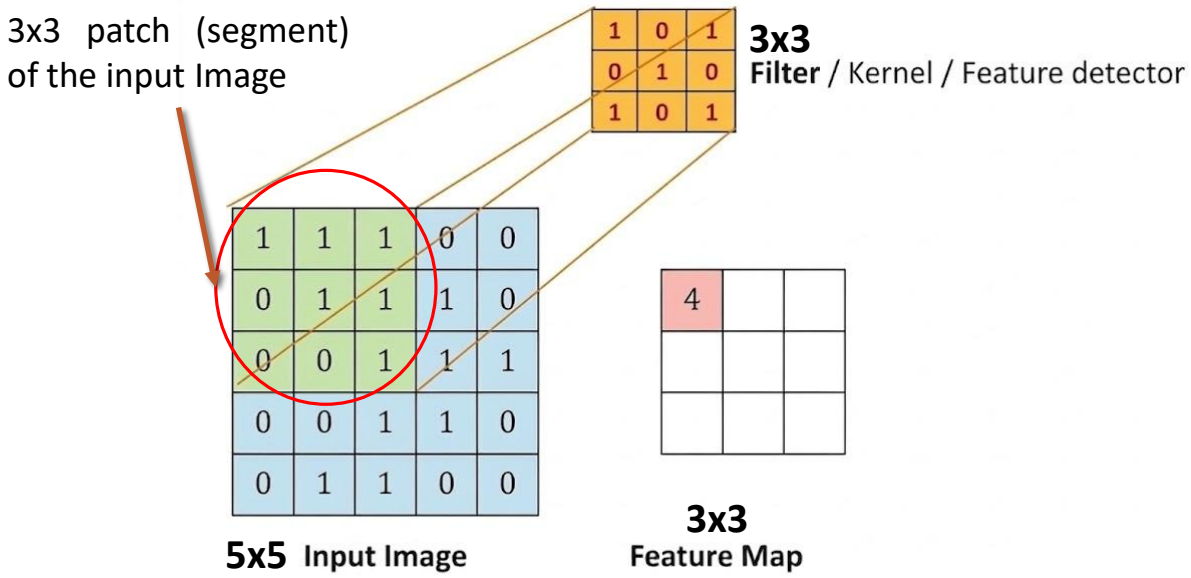
- **Action: Reduces the resolution of the feature maps (Max Pooling, Average Pooling).**
- **Purpose: Decreases computational load and makes the feature detection robust against small translations or distortions.**

4. Classification (Fully Connected Layer)

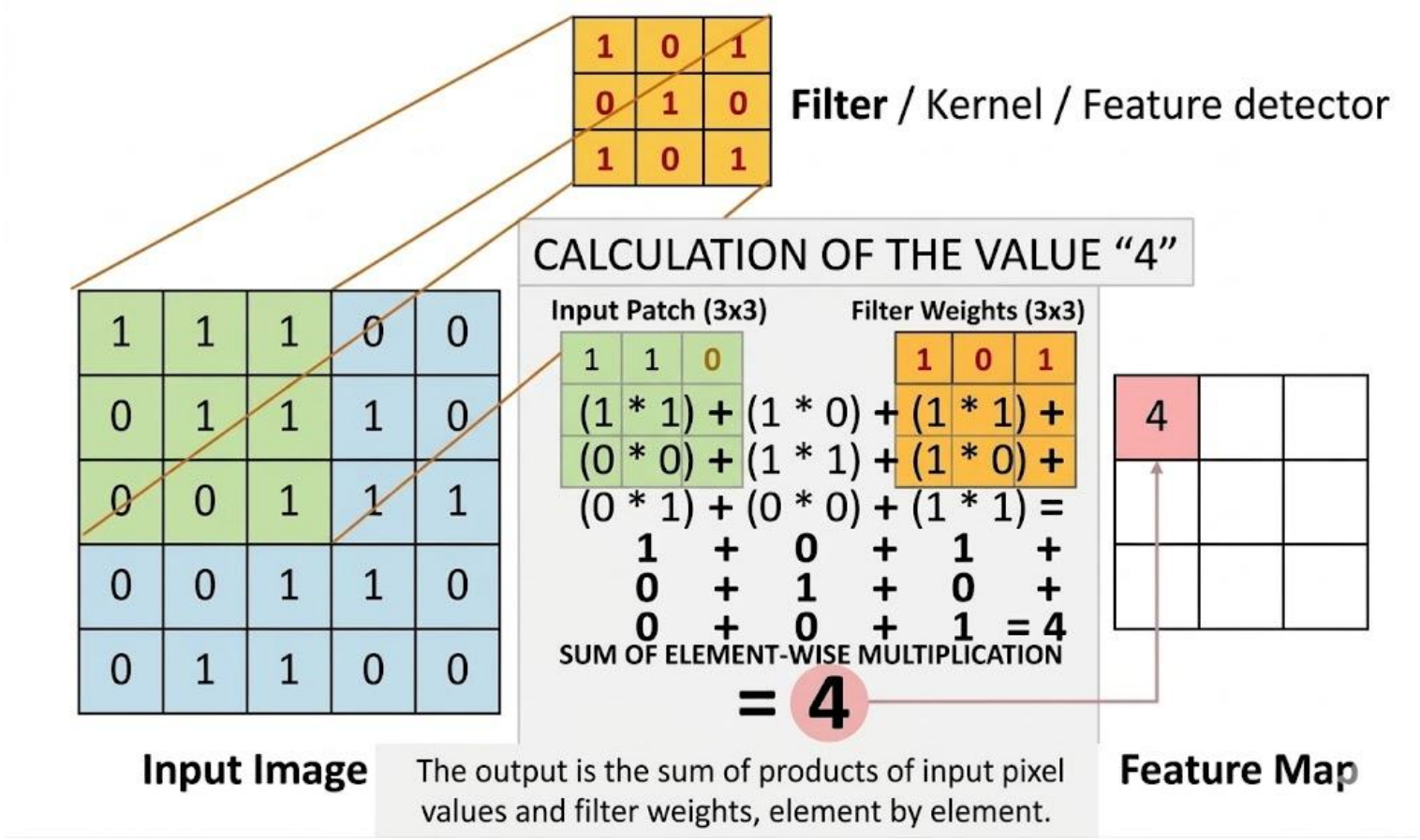
- **Action: Connects every neuron from the previous layer to every neuron in the output layer ANN.**
- **Purpose: Combines all high-level features to calculate the final probability scores for each class.**

Core Functions of Convolutional layer

- **Feature Extraction→** Converts raw pixel data into high-level spatial representations.
- **Filter Application→** Utilizes various filters (kernels or feature detectors) to generate corresponding feature maps.
- **Automated Learning→** The network identifies optimal filter values independently during the training process through backpropagation, rather than using manually defined hyperparameters.
- **Diverse Representation→** Increasing the number of filters enables the extraction of a broader range of image features.
- **Enhanced Recognition→** Improved feature diversity allows the network to recognize complex patterns and generalize more effectively to unseen data.



Element-wise Multiplication in Convolutional Layers



Element-wise Multiplication in Convolutional Layers

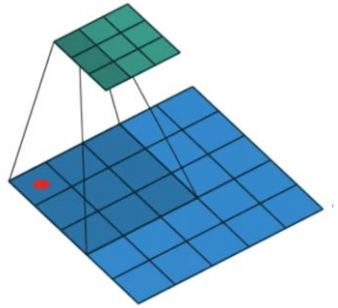
- The size of the output volume is controlled by three parameters that we need to decide before the convolution step is performed:
  - ✓**Depth:** Depth corresponds to the number of filters we use for the convolution operation.
  - ✓**Stride:** Stride is the number of pixels by which we slide our filter matrix over the input matrix.
  - ✓**Zero-padding:** Sometimes, it is convenient to pad the input matrix with zeros around the border, so that we can apply the filter to bordering elements of our input image matrix.
    - With zero-padding *wide convolution*
    - Without zero-padding *narrow convolution*

Illustrative Computation of Convolutional Layer

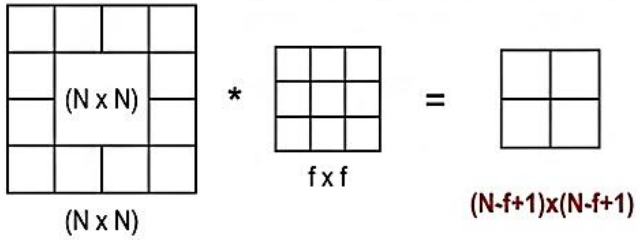
If we apply filter f x f to a 2D input matrix, the output dimension depends on padding (assuming stride=1):

Case 1: Without Padding (Valid Convolution)

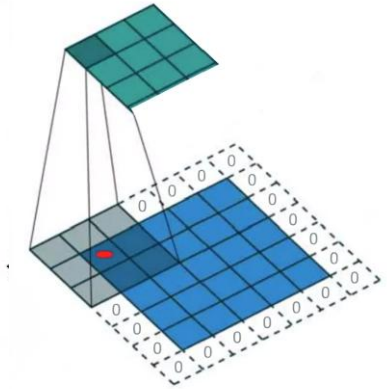
If we have N x N image size and f x f filter size then after convolution result will be
(N x N) * (f x f) = (N-f+1)x(N-f+1)



Schematic: (4x4) * (3x3) = (2x2)



Case 2: With Padding (Same Convolution)

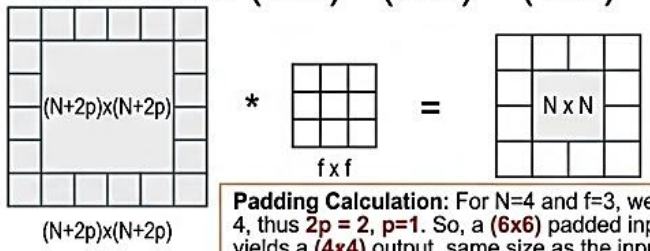


If we apply filter f x f in (N+2p) x (N+2p) input matrix with padding p, then we will get output matrix dimension of **(assuming stride = 1):**
(N+2p-f+1) x (N+2p-f+1)

As after applying padding the same dimen as original input dimension (N x N) should be generated, hence:

→ (N+2p-f+1) x (N+2p-f+1) equivalent to NxN (N+2p-ft) = N), so [**p = (f-1)/2**]

Schematic: (6x6) * (3x3) = (4x4)

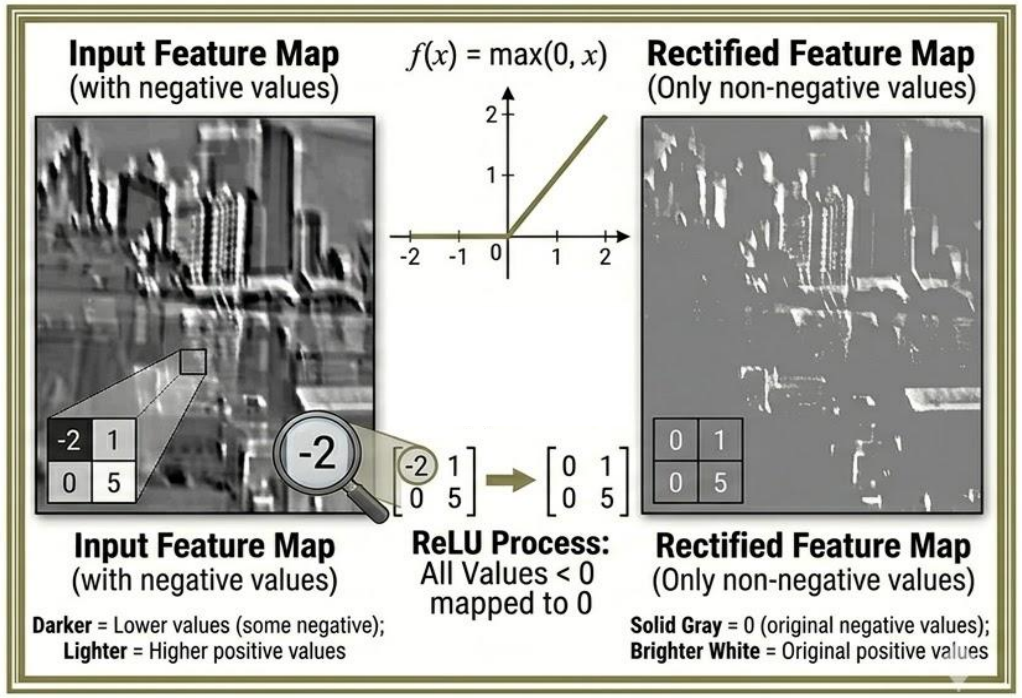


Padding Calculation: For N=4 and f=3, we solve (4+2p-3+1) = 4, thus **2p = 2, p=1**. So, a **(6x6)** padded input with a 3x3 filter yields a **(4x4)** output, same size as the input.

Non-Linearity
Rectified Linear Unit (ReLU)

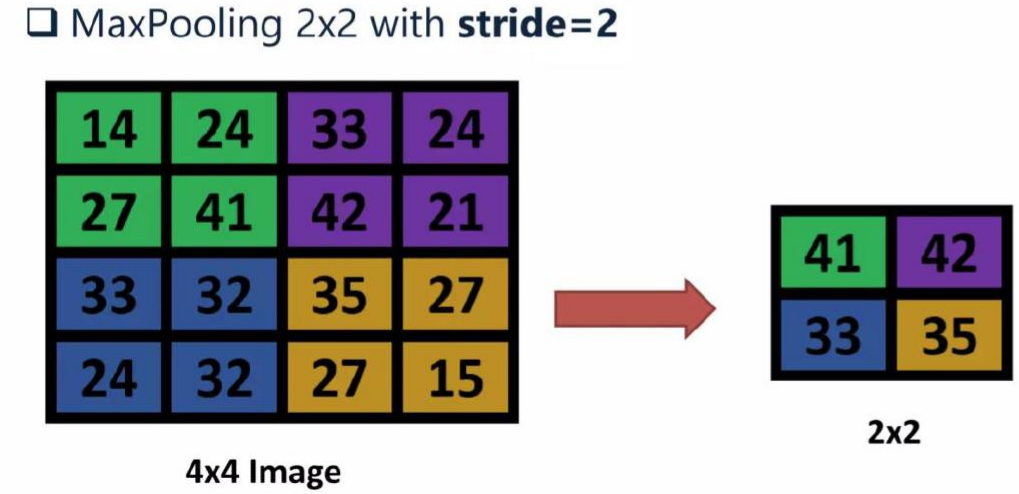
A Rectified Linear Unit (ReLU) is an activation function that introduces the property of non-linearity to a deep learning model and solves the vanishing gradients issue.

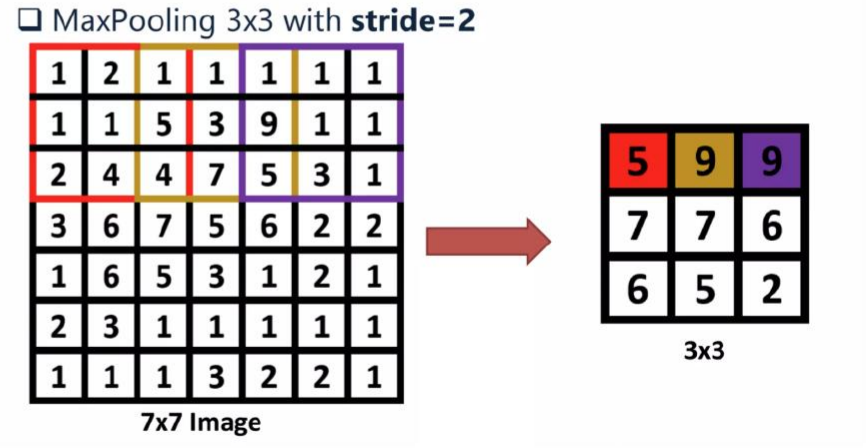
- **ReLU** is an element-wise operation (applied per pixel).
- **ReLU** replaces all negative pixel values in the feature map with zero.
- **Convolution** is a linear operation involving element-wise matrix multiplication and addition.



Pooling Layer

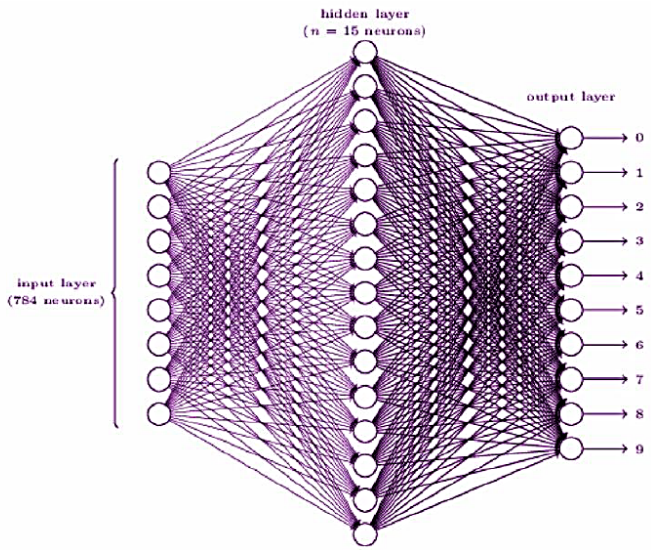
Reduces the dimensionality of each feature map but retains the most important information. Pooling can be of different types: Max, Average



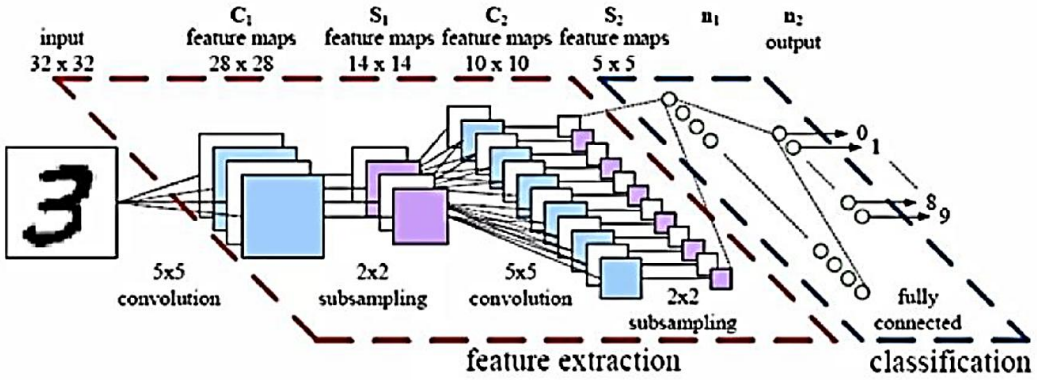


Fully Connected (FC) Layer

- A traditional Multi-Layer Perceptron.
- The term “Fully Connected” implies that every neuron in the previous layer is connected to every neuron on the next layer.
- Their activations can hence be computed with a matrix multiplication followed by a bias offset.
- The purpose of the Fully Connected layer is to use the high-level features for classifying the input image into various classes based on the training dataset.



Softmax
is an activation function that is typically used in the last layer of a neural network to predict the class of an input image.



Dropout Layer

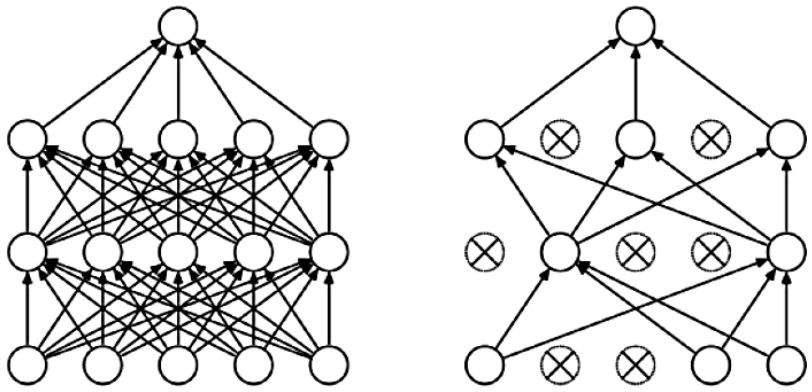
The Problem of Overfitting:

- Deep learning models often overfit by memorizing specific training data features too closely, rather than learning generalizable patterns.
- This leads to exceptional performance on training sets but poor accuracy on new, unseen test data.

The Solution: Dropout Layers

→ A mechanism of randomly drop (deactivate) a set of input and hidden nodes during each training iteration.

→ Temporarily removes all forward and backward connections for dropped nodes, creating a fraction of nodes to be dropped, determined by a dropout probability (*p*).



(a) Standard Neural Net (b) After applying dropout.

The dropout technique helps with the overfitting, specially on dense layers. Drop occur *only at training time*, not on test time.